



Agentic AI Systems: METIS and HedraRAG

March 31, 2026

Nidhil Nayudu, Dillan Morell, Téa Hajratwala



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation

Siddhant Ray, Rui Pan, Zhuohan Gu et al.

SOSP 2025



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

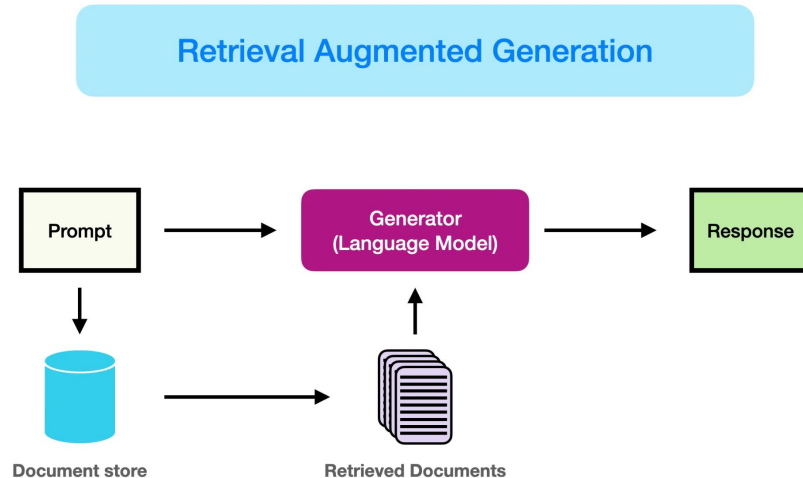
Background

What is RAG?

RAG = Retrieval Augmented Generation:

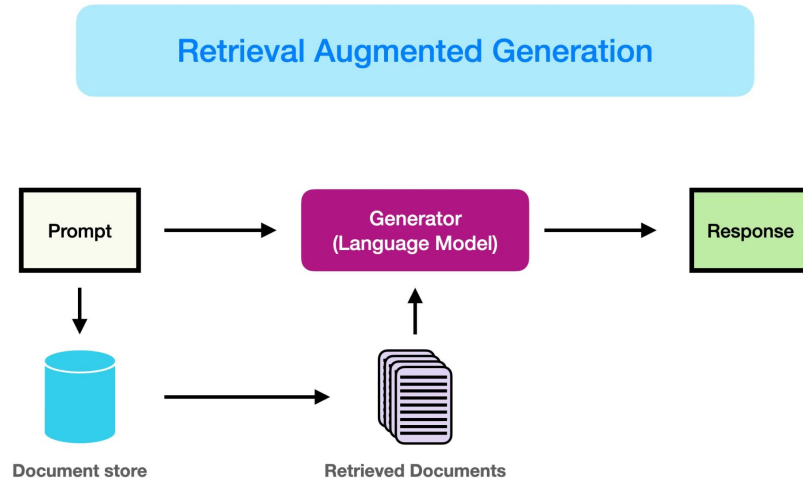
Augmenting prompts with external knowledge to generate high quality responses.

Use Case: Q & A Workflows that require data unavailable to the LLM



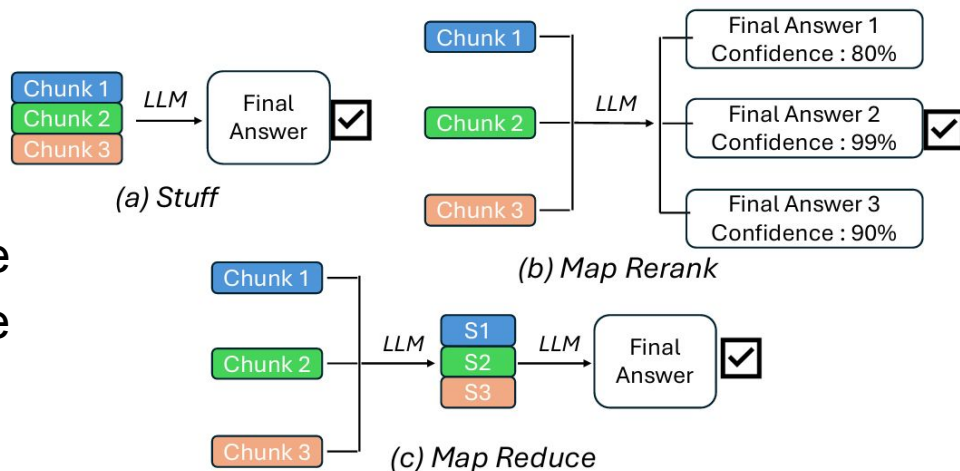
RAG Workflow

- **Setup:** store document chunks as embeddings in a vector database
- **Retrieve:** Find the chunks most similar to our prompt embedding by measuring vector similarity
- **Synthesize:** Combine the chunks to form a new query that gets sent to the LLM



Synthesis Methods

- (a) **Stuff:** Submit all chunks in one query
- (b) **Map Rerank:** Submit separate queries for each chunk and choose answer with the highest confidence
- (c) **Map Reduce:** Summarize each chunk individually and submit all summaries in one query



Goal of RAG Systems

- Prior work chooses one goal:
 - **Minimize delay** via query scheduling (i.e. Parrot OSDI'24)
 - **Maximize quality** via tuning (i.e. AdaptiveRAG ACL'24)
- Difficult to do both since higher quality responses often require more computation leading to longer delays
- METIS optimizes the tradeoff between the quality and latency



RAG Configuration Knobs

Changing each knob comes with trade-offs

How many chunks to retrieve?

Too few chunks: faster synthesis and generation, but risks less accuracy

Too many chunks: slower synthesis and generation, but potentially more accuracy



RAG Configuration Knobs

Changing each knob comes with trade-offs

How many chunks to retrieve?	How to synthesize chunks?
Too few chunks: faster synthesis and generation, but risks less accuracy	Map Rerank: fast because of less computation, but risks low accuracy
Too many chunks: slower synthesis and generation, but potentially more accuracy	Stuff: slower than map rerank, but offers potentially better accuracy up until a point (<i>lost-in-the-middle-problem</i>). Map Reduce: slower because of summarization, but offers better context quality than stuff .

RAG Configuration Knobs

Changing each knob comes with trade-offs

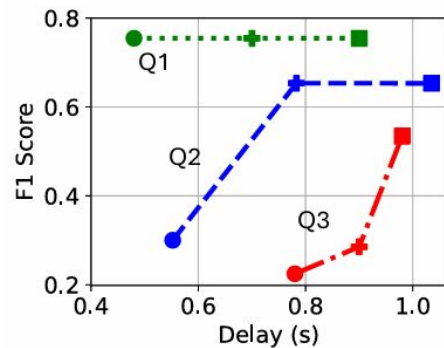
How many chunks to retrieve?	How to synthesize chunks?	How much to summarize?
Too few chunks: faster synthesis and generation, but risks less accuracy Too many chunks: slower synthesis and generation, but potentially more accuracy	Map Rerank: fast because of less computation, but risks low accuracy Stuff: slower than map rerank, but offers potentially better accuracy up until a point (<i>lost-in-the-middle-problem</i>). Map Reduce: slower because of summarization, but offers better context quality than stuff.	Short summaries: faster to generate, risk losing more information Long summaries: slower to generate, but retain more information



Problem

RAG Quality-Latency Tradeoffs Vary per Query

- Simple queries don't benefit from complex synthesis methods, whereas complex queries do



(a) Change: synthesis method from map_rerank (circle), stuff (plus) and map_reduce (square)

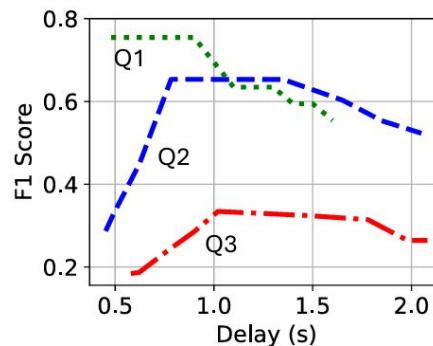
Q1: In what county was William W. Blair born

Q2: Are Alison Skipper, Diane Gilliam Fisher, and Rachel McAdams from the same country?

Q3: When and why did the Voyager 1, the spacecraft that detected storms on Neptune, leave our solar system?

RAG Quality-Latency Tradeoffs Vary per Query

- Simple queries need fewer chunks
- Complex queries need more chunks



(b) Change: Increase # of chunks
from 1 to 35 with stuff (left to right)

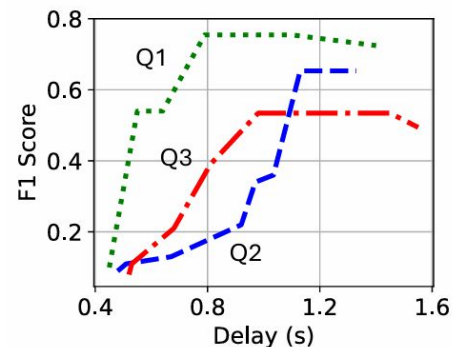
Q1: In what county was William W. Blair born

Q2: Are Alison Skipper, Diane Gilliam Fisher, and Rachel McAdams from the same country?

Q3: When and why did the Voyager 1, the spacecraft that detected storms on Neptune, leave our solar system?

RAG Quality-Latency Tradeoffs Vary per Query

- Simple queries only need a short summary
- Complex queries benefit from a longer, more in-depth summary



(c) Change: Increase intermediate length from 1 to 100 with map_reduce (left to right)

Q1: In what county was William W. Blair born

Q2: Are Alison Skipper, Diane Gilliam Fisher, and Rachel McAdams from the same country?

Q3: When and why did the Voyager 1, the spacecraft that detected storms on Neptune, leave our solar system?

Static Configurations have an Upper Bound

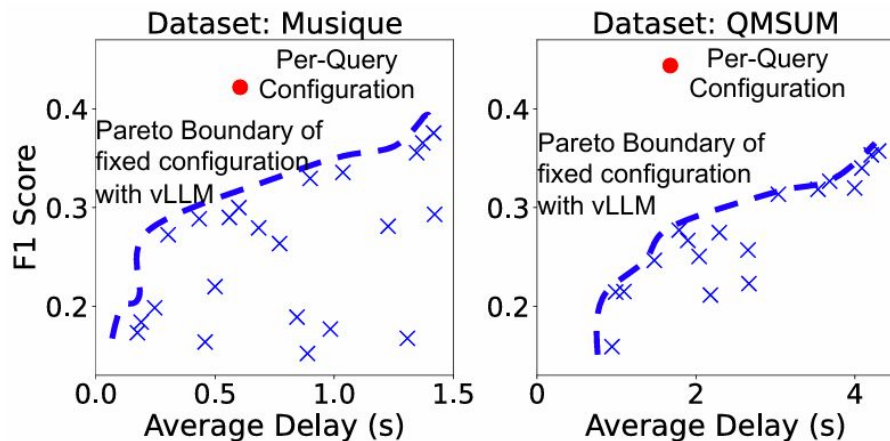


Figure 5. Per-query configuration can achieve significantly better quality-delay tradeoffs across queries compared to every fixed configuration choice.



Combinatorial Challenge

- Tuning configurations across multiple knobs creates an exponentially large search space
- Profiling all configurations is expensive and slow

$$\begin{array}{ccccccc} 30 & \times & 50 & \times & 3 & = & \mathbf{4500} \\ \text{(chunk values)} & & \text{(summary lengths)} & & \text{(synthesis methods)} & & \text{(configurations)} \end{array}$$

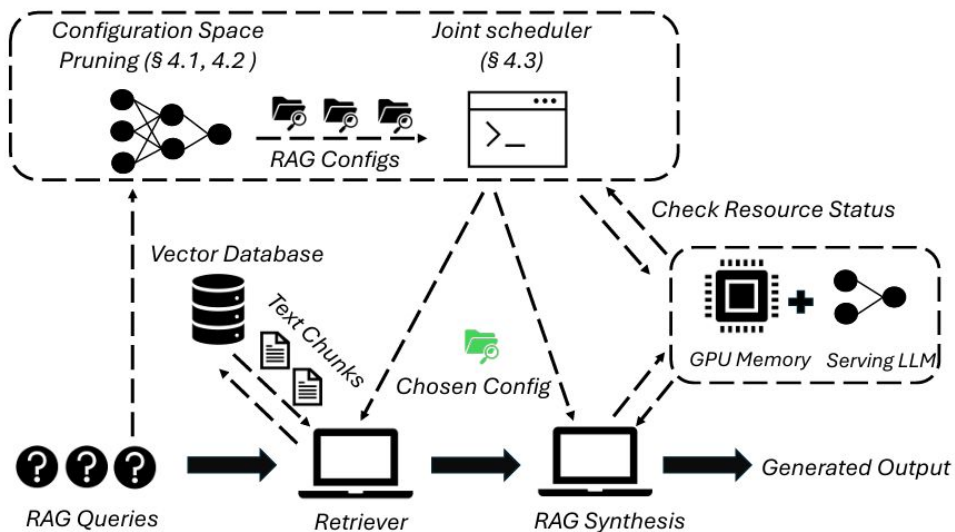


MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Solution

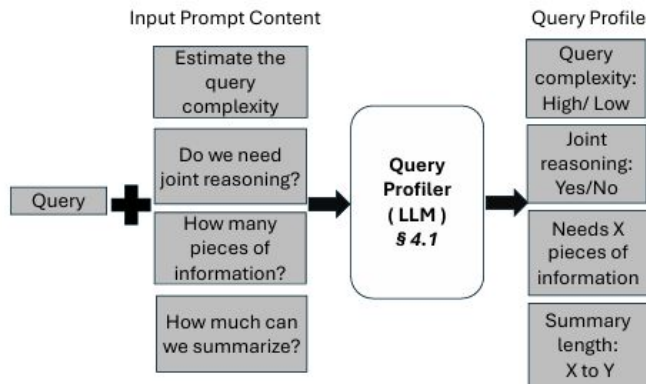
METIS Solution

- **Phase 1:** Prune the configuration space (focus on high accuracy)
- **Phase 2:** Joint optimization (focus on response delay and GPU memory)

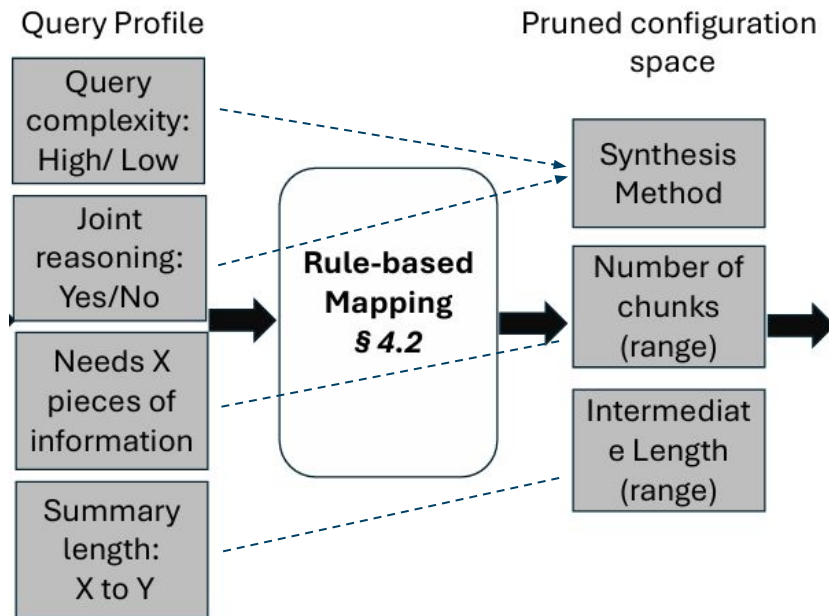


Step 1: Query Profiler

- METIS sends the incoming query to a lightweight LLM
- Analyzes the query alongside the database metadata (chunk size, topic summaries)
- Outputs four dimensions
 - a. Query Complexity: High/Low
 - b. Joint Reasoning: Yes/No
 - c. Pieces of Information: 1-10 facts required
 - d. Summary Length: 30-200 words

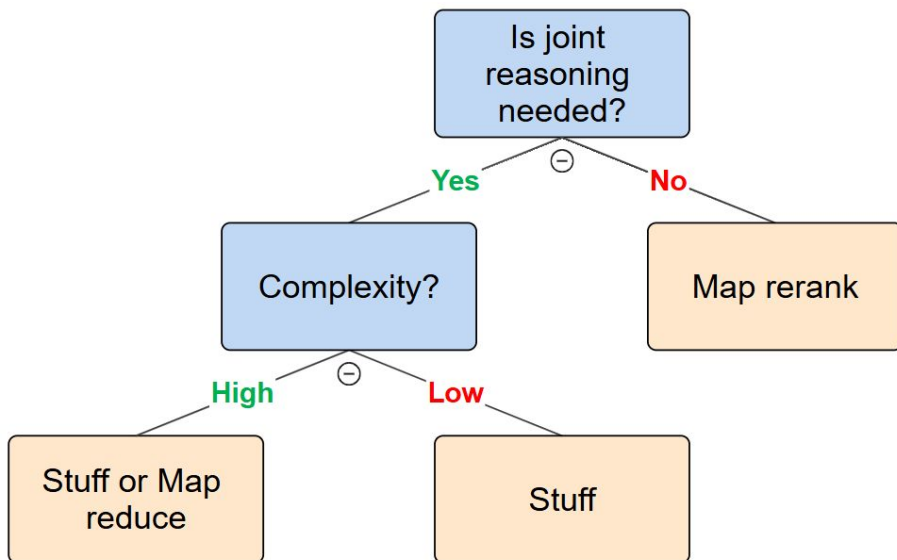


Step 2: Rule-Based Mapping



Step 2: Rule-Based Mapping

Synthesis Method



Number of Chunks

- Profiler identifies n pieces of information
- The pruned range is restricted to n to $3n$ chunks

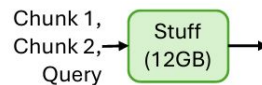
Summary Length

- Pruned range is restricted to min and max output by the profiler

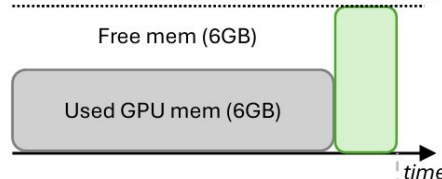
Step 3: Resource-Aware Scheduling

- METIS calculates the GPU cost of every pruned configuration
- Selects the highest quality configuration that fits into the current GPU batch

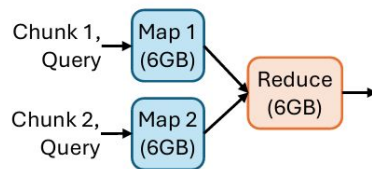
In general, "Stuff" is faster than "MapReduce" as a RAG config



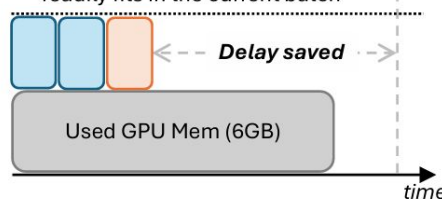
Yet, "Stuff" is memory-intensive and thus is slower when available GPU RAM is limited



(a) Baseline Separates configuration selection and scheduling



We select MapReduce as it can readily fits in the current batch



(b) Ours performs configuration selection and scheduling jointly

System Safeguards and Fallbacks

- **Confidence Thresholds (Step 1: Query Profiler)**
 - LLM's confidence score drops below 90%
 - Ignore the profiler and use configurations from the last 10 successful queries
- **Graceful Fallback (Step 3: Scheduling)**
 - Server is overloaded and none of the pruned options fit
 - Fall back to cheap configuration to guarantee low latency



Results

Results vs. Baseline

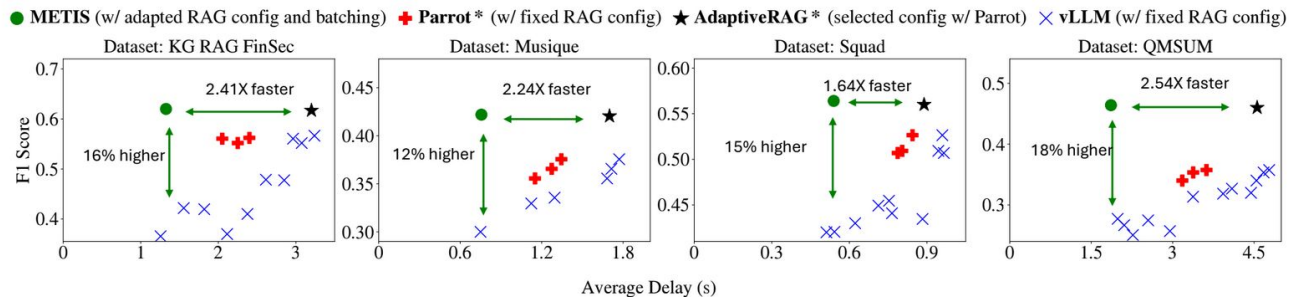


Figure 10. METIS achieves 1.64 – 2.54× lower delay compared to both best fixed configuration baselines and quality-optimized RAG configuration without sacrificing generation quality.

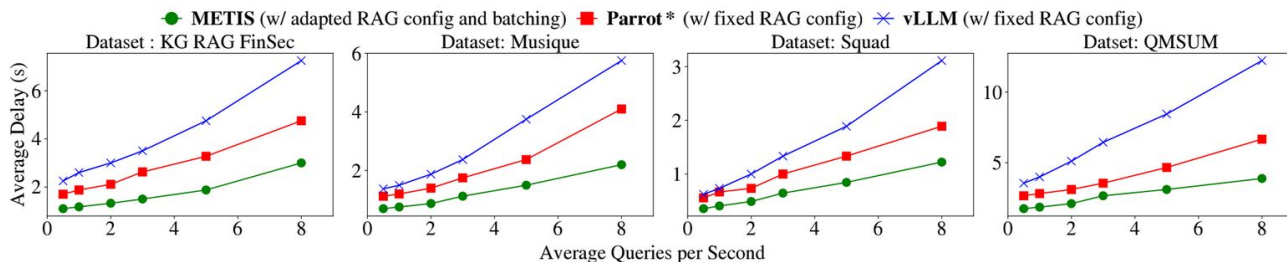


Figure 11. METIS achieves 1.8 – 4.5× higher throughput (at 1.8 seconds) than baselines which use fixed configurations of closest (not higher) quality.

Results vs. Baseline

- 1.64–2.54x Lower delay
- 1.8 – 4.5x Higher throughput
- 2.38 – 6.8x Lower cost
- Similar/Higher quality
 - Adapts the RAG config per query

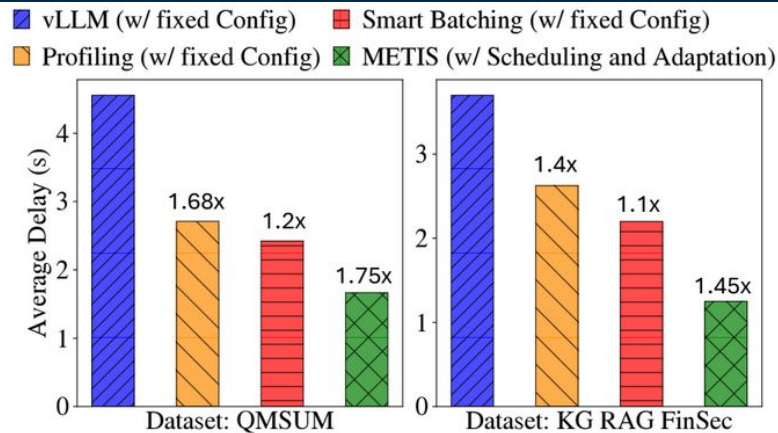


Figure 12. Understanding the delay improvement in METIS

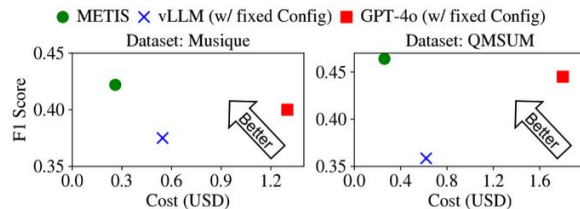


Figure 13. Even with increasing the inference model size, fixed configurations have 2.38 – 6.8x higher cost and lower quality compared to METIS.

Further Results

- Adding the feedback to the profiler increases quality
 - F1-score improves by 4 – 6%

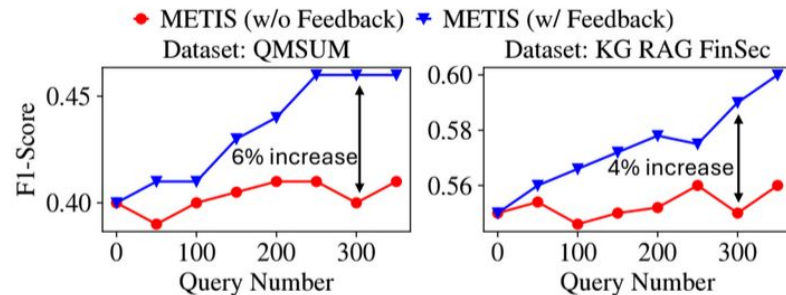


Figure 14. Improvement for METIS using feedback from the output helps improve the F1-score by 4 – 6%.

Further Results

- Tuning RAG-configuration knobs shows 5, 4, 3% higher F1-Score compared to vLLM
- By adding the scheduling, 2.8x lower delay reduction in delay

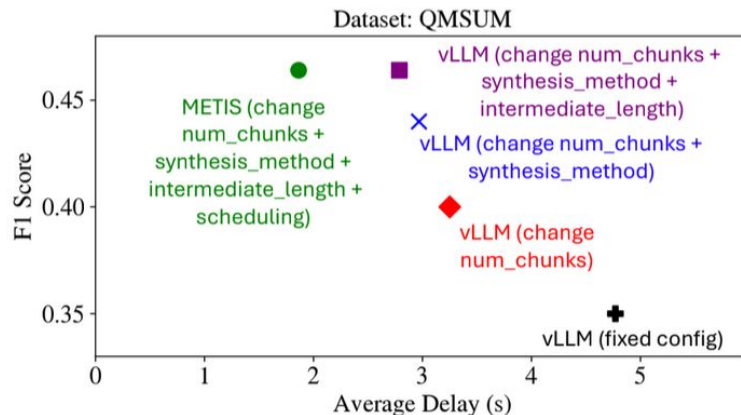


Figure 16. Breakdown analysis: By tuning more knobs in METIS, we can see better quality-delay tradeoffs.



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Conclusion



Strengths

- By adapting knobs like chunk count and synthesis method per-query, it avoids the inefficiencies of static configurations
- Uses a lightweight LLM profiler to estimate query complexity, reducing a massive combinatorial configuration space
- Jointly considers configuration and GPU availability ensuring that memory-heavy choices do not bottleneck the scheduler queue



Weaknesses

- The LLM-based query profiler introduces an initial latency penalty (roughly 10% of execution time)
- Struggles with vague prompts where the LLM profiler cannot estimate required reasoning depth or chunk count
- Relies on a rigid, rule-based mapping to translate LLM profile estimations into configurations
 - Not suitable for non-Q&A tasks



Conclusion

- **Idea:** The first system to focus on optimizing tradeoffs between response delay and generation quality in RAG
 - Jointly schedules RAG queries and adapts key configurations on a per-query basis
- **Result:** It outperforms the state-of-the-art, reducing generation latency without compromising response quality

Future Work

- Extending the dynamic tuning beyond chunk count and synthesis methods
 - Selection of the optimal embedding model, retrieval index, or even the underlying serving LLM itself
- Leveraging multi-turn chat history from the same user
 - Helps profile and answer highly under-specified queries
- Further exploring how different embedding models impact the profiler's performance and query clustering
- Adapt their scheduler for complex, multi-step agentic RAG



HedraRAG: Co-Optimizing Generation and Retrieval for Heterogeneous RAG Workflows

Zhengding Hu, Vibha Murthy, Zaifeng Pan et al.

SOSP 2025



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Background



Heterogenous RAG Workflows

One-shot

Simple question

Who got the first Nobel prize in physics?



Doc: First Nobel Prize winner is ...

Answer: Wilhelm Röntgen.

1 retrieval
1 generation

Multistep

Multi-hop reasoning

When did Li Longji's father die?



Subquestion: Who is did Li Longji's father

Doc1

Subquestion: When did Li Dan die?

Doc2

Answer: 716 AD.

2 retrieval
3 generation

Iterative

Knowledge summary

Discovery of penicillin and subsequent impact.



Question: How is penicillin discovered.

Doc1

Penicillin was discovered by ...

Doc2

In 1940s, a team at the Oxford ...

Doc3

... → **Doc4** → ... →

>3 retrieval
>3 generation

Heterogenous RAG Workflows

then

One-shot

Simple question

Who got the first Nobel prize in physics?

Doc: First Nobel Prize winner is ...

Answer: Wilhelm Röntgen.

As prompts increase in complexity, so do RAG workflows!

now

Multistep

Multi-hop reasoning

When did Li Longji's father die?

Subquestion: Who is did Li Longji's father

Doc1

Subquestion: When did Li Dan die?

2 retrieval
3 generation

Doc2

Answer: 716 AD.

now

Iterative

Knowledge summary

Discovery of penicillin and subsequent impact.

Question: How is penicillin discovered.

Doc1

Penicillin was discovered by ...

>3 retrieval
>3 generation

Doc2

In 1940s, a team at the Oxford ...

Doc3

... → Doc4 → ... →



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Motivation



Problems with Existing Work

1. Generation and retrieval are solved separately, causing pipeline bubbles
2. Accumulated latency in multi-round workflows
3. With large vector databases, retrieval can be slow and cause a bottleneck
4. How to abstract the heterogenous RAG workflows?

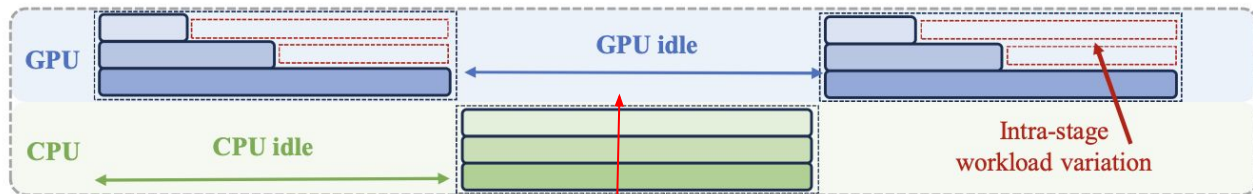
HedraRAG addresses all of these! (see next)



Problem #1: Generation and retrieval are solved separately, causing pipeline bubbles

Idea #1: Parallelize independent generation and retrieval stages

Generation typically uses different hardware than retrieval (GPU vs CPU).



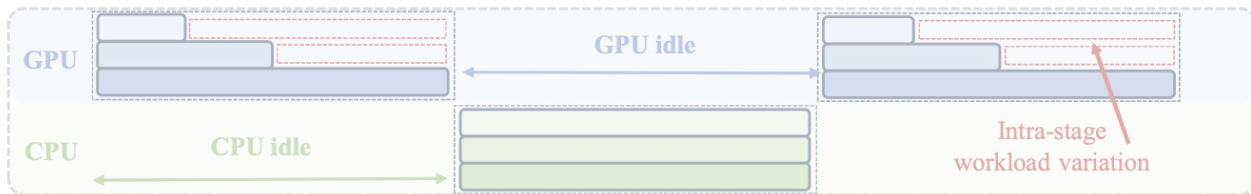
(a) **Modular** and **Sequential** stages (General RAG frameworks)

Bad: CPU and GPU underutilized!

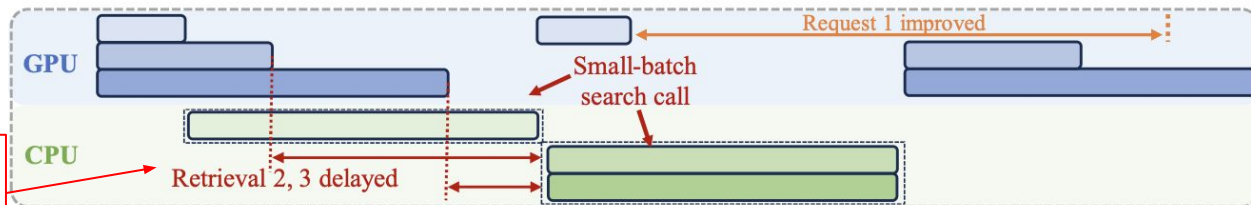
Problem #1: Generation and retrieval are solved separately, causing pipeline bubbles

Idea #1: Parallelize independent generation and retrieval stages

vLLM + FAISS improve throughput, but resources somewhat underutilized



(a) **Modular and Sequential** stages (General RAG frameworks)



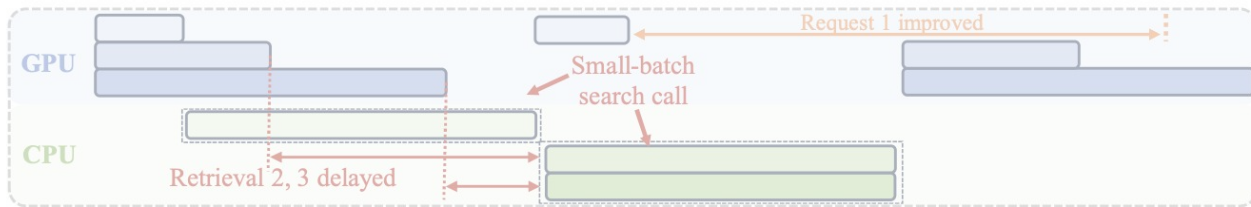
(b) **Dynamic-batched** generation stages + **Fixed-batched** retrieval stages (vLLM + Faiss)

Fixed retrieval batches means 2, 3 delayed

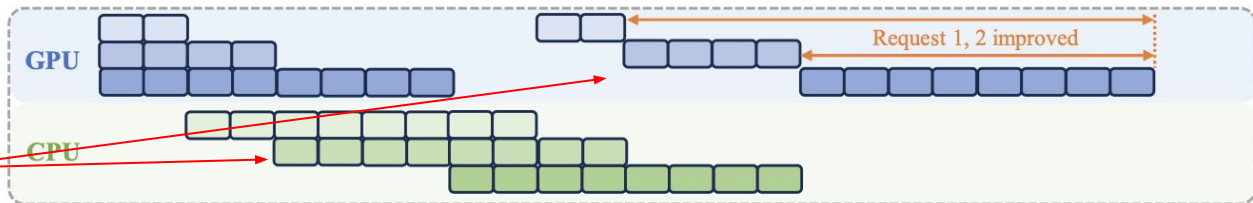
Problem #1: Generation and retrieval are solved separately, causing pipeline bubbles

Idea #1: Parallelize independent generation and retrieval stages

Introduce fine grained sub stage partitioning for both generation and retrieval



(b) **Dynamic-batched** generation stages + **Fixed-batched** retrieval stages (vLLM + Faiss)



(c) **Fine-grained sub-stage** partitioning and **Dynamic-batched** pipelining (HedraRAG)

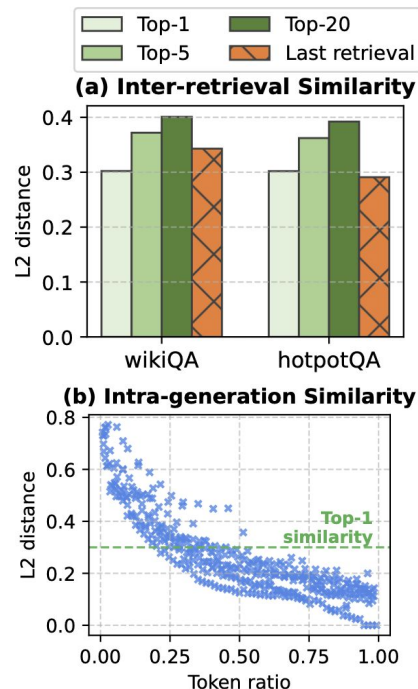
Problem #2: Accumulated latency in multi-round workflows

Idea #2: Utilize Semantic Similarity

Heterogenous RAG workflows often have many rounds of generation + retrieval, often with similar context.

Inter-Retrieval Similarity: Adjacent retrievals will likely get similar documents (so let's cache them).

Intra-Generation Similarity: Partial generations are mostly accurate, and can be used to start next retrieval.





Problem #2: Accumulated latency in multi-round workflows

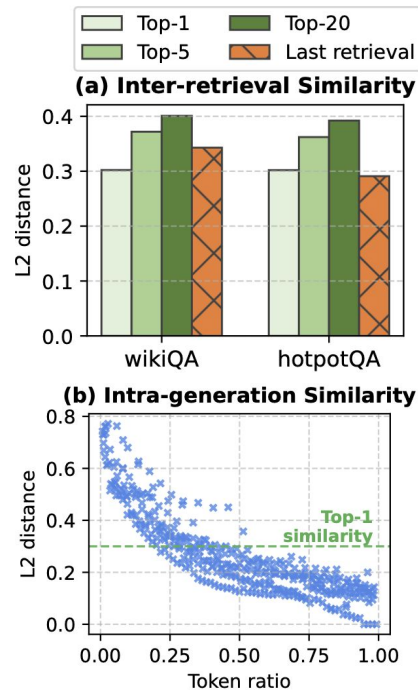
Idea #2: Utilize Semantic Similarity

Heterogeneous
generation

Inter-Retrieval
same use
them).

Takeaway: Use this knowledge to optimize intra-request generation and retrieval!

Intra-Generation Similarity: Partial generations are mostly accurate, and can be used to start next retrieval.





Problem #3: With large vector databases, retrieval can be slow and cause a bottleneck

Idea #3: Utilize Inter-Request Retrieval Skewness

Observation: In real workloads, vector retrieval is not uniform—it is concentrated in ‘hotspot clusters’.

Problem: Hotspot clusters are not static!

Solution: Need to asynchronously cache + update which clusters are hot.

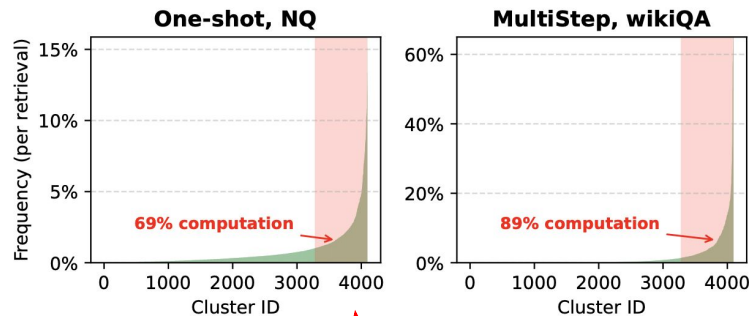


Figure 8. Skewness of access frequency among clusters when using IVF4096 indexing. Cluster IDs are sorted in descending order of frequency.

The top 20% of hotspot clusters account for over 69% of total computation



Problem #3: With large vector databases, retrieval can be slow and cause a bottleneck

Idea #3: Utilize Inter-Request Retrieval Skewness

Another Problem: using the CPU to retrieve indexes is slow due to PCIe bandwidth.

Final Solution: Use the GPU to store hot clusters, which are asynchronously updated based on runtime.

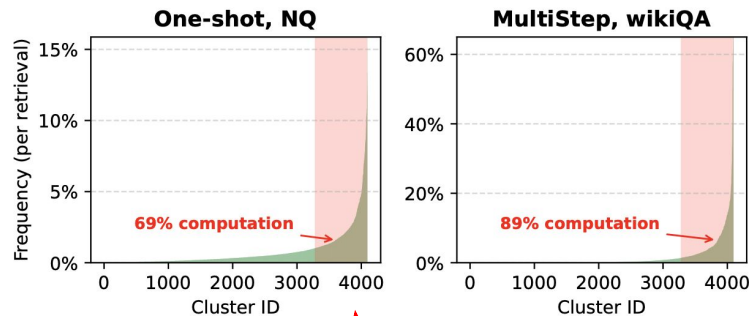


Figure 8. Skewness of access frequency among clusters when using IVF4096 indexing. Cluster IDs are sorted in descending order of frequency.

The top 20% of hotspot clusters account for over 69% of total computation



Problem #3: How to abstract heterogenous RAG workflows?

Idea #4: Use a graph based approach to customize and optimize easily.

Solution: RAGraph! (see design section)



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Design

Contribution #1: RAGraph

A graph-based abstraction for RAG workflows.

Components:

- Generation Nodes
- Retrieval Nodes
- Edges

Listing 1. Construct RAG workflows with graph primitives.

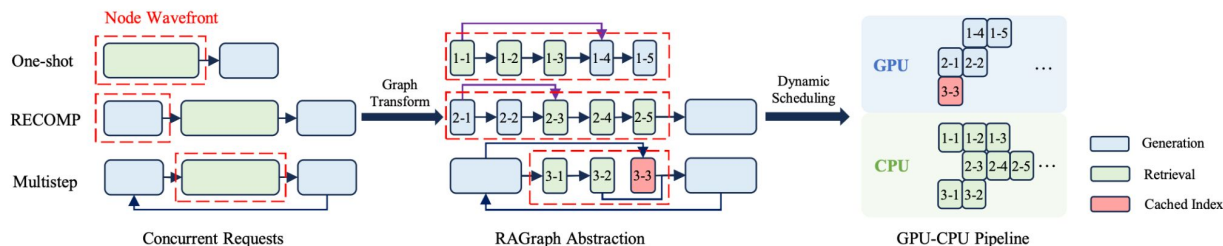
```
1 from HedraRAG import RAGraph, START, END
2 from HedraRAG import Server
3 # HyDE-style workflow
4 g1 = RAGraph()
5 g1.add_generation(0, prompt="Generate a hypothesis
6                       for {input}.", output="hypopara")
7 g1.add_retrieval(1, topk=5, query="hypopara", output="docs")
8 g1.add_generation(2, prompt="Answer {query} using {docs}.")
9 g1.add_edge(START, 0); g1.add_edge(0, 1)
10 g1.add_edge(1, 2); g1.add_edge(2, END)
11 # Multistep-style workflow
12 g2 = RAGraph()
13 g2.add_generation(0, prompt="Decompose {input} into
14                       subquestions.", output="subquestion")
15 g2.add_retrieval(1, topk=2, query="subquestion",
16                       output="docs")
17 g2.add_generation(2, prompt="Answer {subquestion}
18                       using {docs}.")
19 g2.add_edge(START, 0); g2.add_edge(0, 1); g2.add_edge(1, 2)
20 g2.add_edge(2, lambda s: 1 if s.get("subquestion") else END)
21 # Server initiating and execution
22 s = Server(generator="Llama3-8B", index="IVF4096")
23 s.add_request("What is RAG?", g1)
24 s.add_request("Compare RAG with long-context models.", g2)
```

Contribution #1: RAGraph

A graph-based abstraction for RAG workflows.

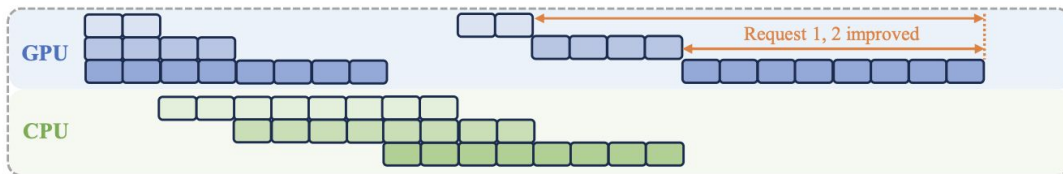
Operations:

- Node Splitting
- Reordering
- Dependency Rewiring



Essentially allows the original workflow to be optimized as needed.

Contribution #2: Fine Grained Sub-Stage Pipelining



(c) Fine-grained sub-stage partitioning and Dynamic-batched pipelining (HedraRAG)

$$mb = \operatorname{argmax}(\Delta_l), \Delta_l = \frac{t_{\text{Retrieval}} - mb}{2} - \frac{t_{\text{Retrieval}}}{mb} \beta$$

- Δ_l : expected latency improvement
- β : denotes the CPU overhead of request scheduling
- $t_{\text{Retrieval}}$: denotes the average time of retrieval stage

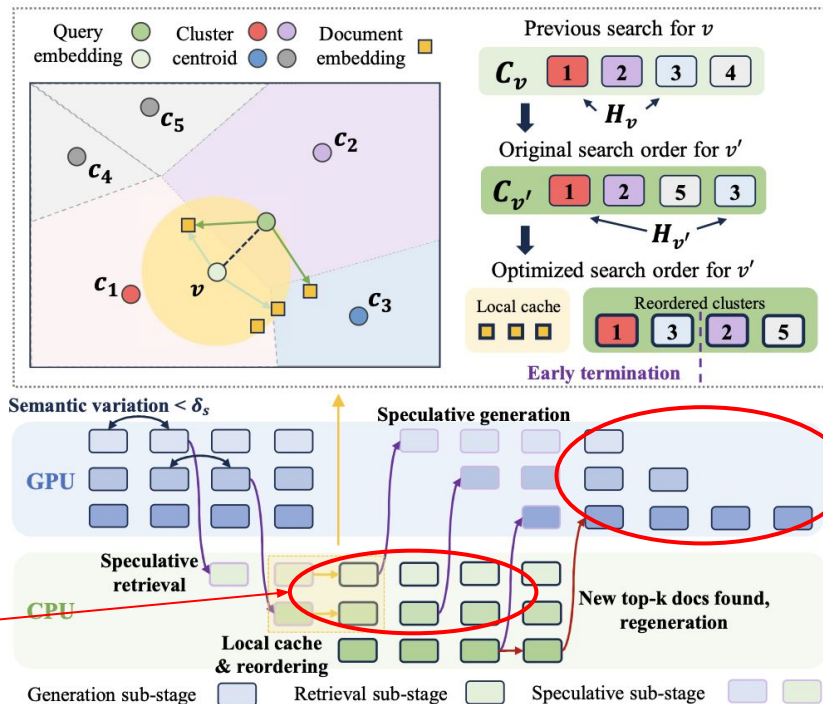
Question: How to keep the right number of generation steps and retrieval clusters?

Answer: Use a time budget!

- Set a max time budget mb . Before a sub stage executes, clusters from incoming retrieval requests are added until the budget is exceeded.

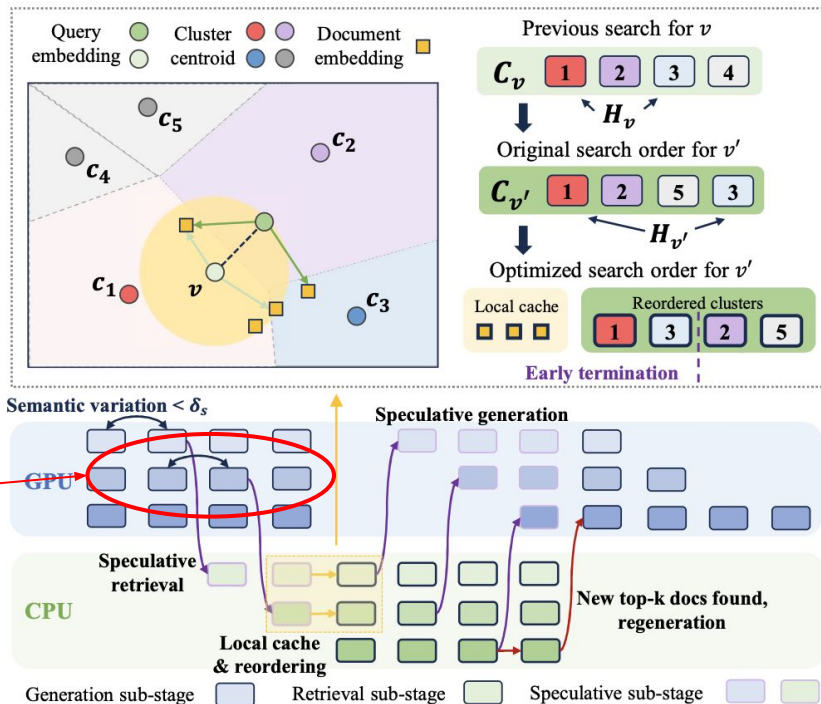
Contribution #3: Similarity-Aware Search Optimization

Speculative Generation



Contribution #3: Similarity-Aware Search Optimization

Speculative Retrieval

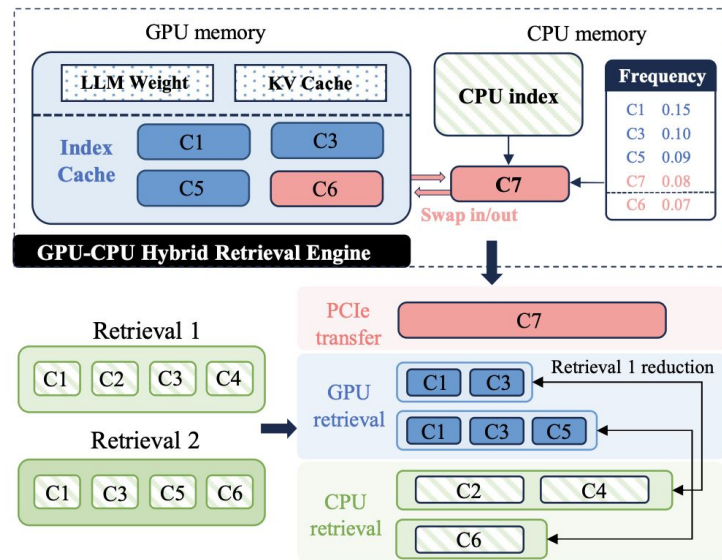


Use partial generation results to start retrieval

Contribution #4: Partial GPU Indexing

1. Leverage index skew by caching hot clusters
 - a. GPU side to avoid PCIe bottleneck
2. If exists in GPU, use it. Otherwise use CPU
3. Monitors which clusters are hot in real time
4. Asynchronously update this cache every 50 sub stages GPU/CPU collaborated vector search

LLM & Index cluster memory coordination





MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Evaluation

Evaluation: Online

HedraRAG works best for more complex workflows

Reduced request latency from **2.2x** to **18.2x** at identical request rates

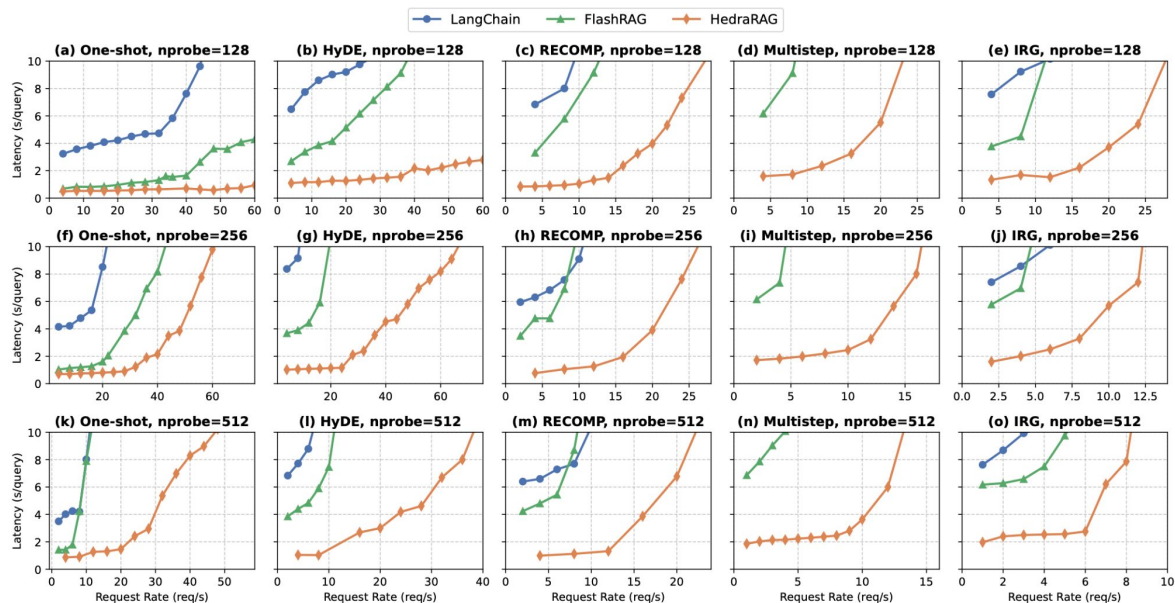


Figure 13. Average request latency when using various RAG workflows.

Evaluation: Offline

Speedup of **3.5x** and **1.3x** over
LangChain and FlashRAG

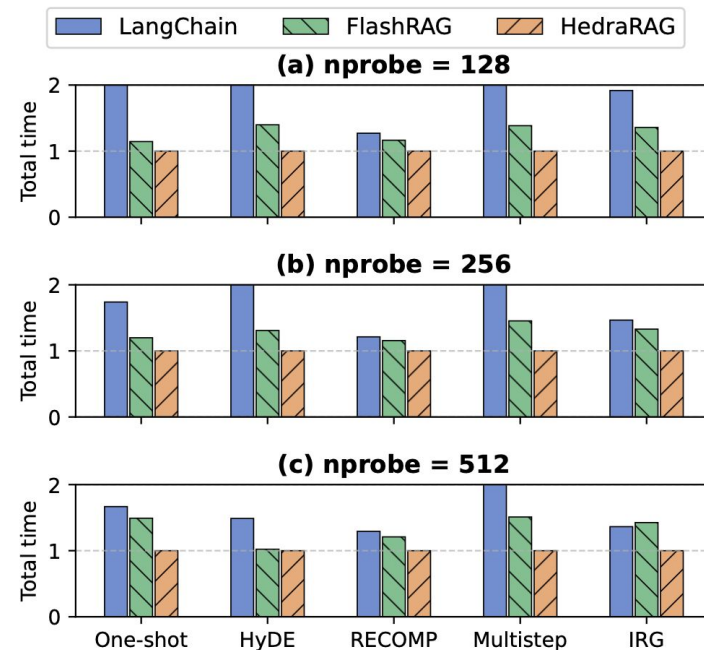


Figure 14. Total offline runtime of different RAG frameworks. All timing results are reported with normalization w.r.t. HedraRAG. For clarity, values over 2 are truncated.

Evaluation: Reordering and Speculation

HedraRAG is consistently the fastest, but not always the most accurate at speculation.

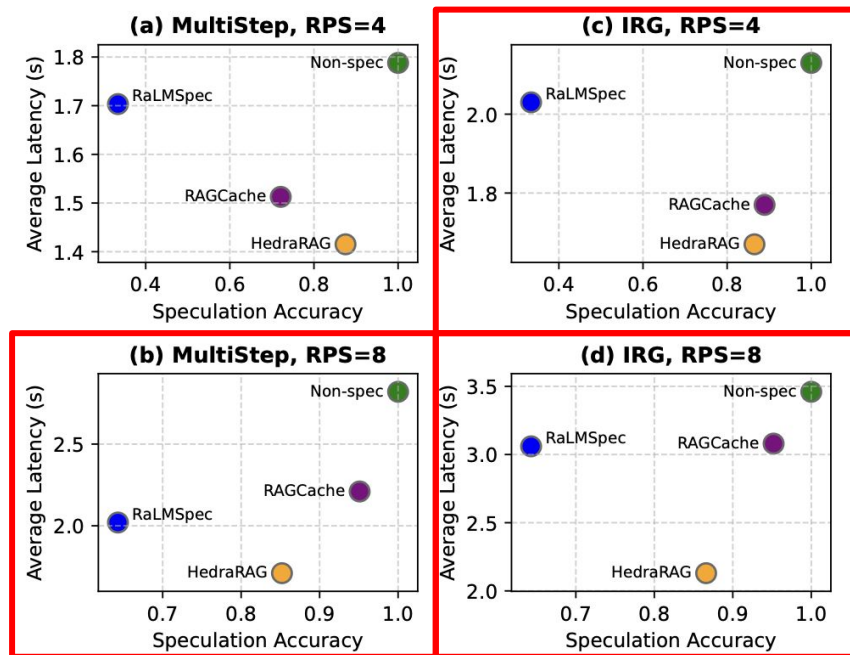


Figure 19. The speculation accuracy and latency comparison across existing speculation methods. RPS: request/second.



Results



Results

Dynamic partitioning and pipelining

- Improves search latency with fine-grained and dynamic batching
 - Minimizes the time for new requests to wait behind long search calls

Reordering and Speculation

- Latency speedup through integration of similarity-aware reordering and a runtime-adaptive speculative execution mechanism
 - Considers RAG workflow heterogeneity and request workload dynamics

Partial GPU Indexing

- Captures skewed index access patterns across concurrent requests by caching frequently accessed clusters



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Conclusion



Strengths

- Effectively minimizes end-to-end wait times by dynamically overlapping database retrieval and LLM generation through pipelining and speculation
- Partial GPU indexing brilliantly capitalizes on hybrid CPU-GPU setups, maximizing throughput without bottlenecking limited VRAM
- Dynamically adjusts to varying request lengths and complex RAG workflows



Weaknesses

- Having a GPU index cache means less space for KV Cache when resources are limited, offline profiling not enough
- Although consistently faster speculation, not all of it is accurate
- Managing asynchronous cache updates, speculative execution rollbacks, and dynamic batching introduces significant engineering overhead



Conclusion

Idea: HedraRAG bridges the gap between high-level RAG heterogeneity and low-level LLM and vector search backends

- Defining transformation operations, semantic-aware speculative execution, and partial GPU index caching

Result: HedraRAG outperforms existing RAG systems across a range of models and work-flows

- Shows the value of rethinking system design to address the challenges of heterogeneous AI pipelines.



Future Work - HedraRAG

Hardware

- Improve PCIe so hot cluster cache can be stored on CPU instead

Software

- Determining KV Cache/hot cluster cache allocation is done with offline profiling. Could be done using a resource manager to be more adaptable at runtime



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

METIS & HedraRAG Comparison

METIS & HedraRAG

Feature	METIS	HedraRAG
Optimization Layer	Application / Workflow Layer	System / Infrastructure Layer
Core Goal	Balance the Quality vs. Delay tradeoff	Maximize Latency & Throughput efficiency
Primary Mechanisms	LLM-based query profiling, rule-based config mapping, joint scheduling	Dynamic pipelining, speculative execution, partial GPU indexing
Adaptation Strategy	Per-Query: Adjusts knobs based on input complexity	Cross-Query: Manages concurrent requests using dynamic batching and vector caching
Key Advantage	Prevents wasting compute on simple queries while ensuring high accuracy for complex ones	Drastically reduces wait times for long search calls and prevents CPU/GPU bottlenecks
Main Overhead	Initial latency penalty from running the LLM profiler before actual RAG execution	High architectural complexity from managing async cache updates and speculation rollbacks